CalendlyByEnzo

Récap complet du projet — construit le 6 avril 2026

8	21	32
Phases complétées	Requirements v1	Fichiers créés

14
Tests unitaires

C'est quoi ce projet ?

CalendlyByEnzo est **ta propre page de prise de rendez-vous**, sur ton domaine (remidene-enzo), avec ton style. Un prospect arrive depuis ton quiz burnout, voit une mini page de vente qui explique ce qu'il va obtenir pendant 30 minutes de mini-coaching, valide qu'il est prêt, choisit un créneau, et booke. Tu reçois une notif Telegram, il reçoit un email de confirmation + un rappel 24h avant.

Le parcours prospect

Quiz Burnout → Page de vente → Checklist → Créneaux → Formulaire → Confirmé !

Les 8 phases expliquées

1 Project Setup INFRASTRUCTURE

En simple : On crée le projet à partir de zéro. C'est comme poser les fondations d'une maison avant de construire les murs. On installe Next.js (le framework web), on configure les couleurs et la police pour que ça ressemble exactement à ton quiz burnout (fond crème, texte sombre, police Inter), et on prépare la base de données.

> **Challenge :** Tailwind CSS version 4 a changé sa façon de fonctionner — plus de fichier de config JavaScript, tout se fait en CSS. Il fallait s'adapter à cette nouvelle syntaxe.

> **Solution :** Utilisation de la directive `@theme` en CSS pour déclarer tous les tokens de design (les 14 couleurs du quiz burnout) directement dans le fichier CSS global. Résultat : les couleurs sont disponibles partout dans l'app via des classes simples comme `bg-bg`, `text-sub`.

Fichiers clés : `globals.css` `layout.tsx` `page.tsx` `db/schema.ts`

2 Availability Engine

LOGIQUE MÉTIER

En simple : C'est le cerveau du système. Une fonction qui prend tes horaires de disponibilité (ex: mardi et jeudi 14h-18h) + tes événements Google Calendar, et calcule les créneaux de 30 minutes réellement disponibles. C'est comme un puzzle : on découpe la journée en tranches de 30min et on enlève celles qui sont déjà prises.

> **Challenge :** Le changement d'heure (été/hiver) en mars et octobre peut décaler les créneaux. Un créneau de 14h peut devenir 15h si on ne gère pas bien les fuseaux horaires.

> **Solution :** Toutes les heures sont calculées en timezone Europe/Paris avec la bibliothèque `date-fns-tz`. 14 tests unitaires vérifient que tout fonctionne : jour normal, jour plein, buffer entre RDV, changement d'heure, délai minimum de 24h, fenêtre maximum de 2 semaines.

Fichiers clés : `lib/availability/engine.ts` `lib/availability/types.ts` `__tests__/engine.test.ts`

3 Google Calendar Integration

API EXTERNE

En simple : On connecte le moteur de disponibilité à ton vrai Google Calendar. Quand un prospect visite la page, le système demande à Google "quand est-ce qu'Enzo est occupé ?" et calcule les créneaux libres. Quand quelqu'un booke, le système crée automatiquement un événement dans ton calendrier.

> **Challenge 1 :** L'authentification Google est complexe (OAuth2, tokens qui expirent, refresh tokens). Et deux personnes pourraient booker le même créneau en même temps (race condition).

> **Challenge 2 :** Turbopack (le compilateur rapide de Next.js) ne supporte pas la librairie de base de données. Le build plantait.

> **Solution :** Un script de setup que tu exécutes une seule fois pour obtenir ton refresh token Google. Pour les bookings simultanés : le système re-vérifie Google Calendar juste avant de créer l'événement — si le créneau a été pris entre-temps, il renvoie une erreur claire. Pour Turbopack : basculé sur Webpack classique pour le build.

Fichiers clés : `lib/google-calendar/client.ts` `api/slots/route.ts` `api/book/route.ts` `scripts/get-refresh-token.ts`

4 Booking Page & Checklist FRONTEND

En simple : La page que le prospect voit. D'abord, une mini page de vente qui explique ce qu'il va obtenir pendant les 30 minutes (clarté, recommandations, prochaines étapes). Ensuite, une checklist : "J'ai 30 minutes", "Je suis dans un endroit calme", "Je suis prêt(e) à parler". Tant que les 3 ne sont pas cochées, impossible de voir les créneaux.

> Challenge : Le ton devait être chaleureux et personnel (coaching, pas vente agressive) tout en restant cohérent avec le style du quiz burnout. C'est un équilibre subtil.

> Solution : Texte en "vous" respectueux, bullet points clairs sur la valeur du coaching, design crème minimaliste avec beaucoup d'espace. Les items de la checklist sont des cartes cliquables avec une animation douce quand on les coche. Le bouton "Voir les créneaux" passe de grisé à actif une fois tout validé.

Fichiers clés : `page.tsx` `components/ChecklistItem.tsx`

5 Slot Picker FRONTEND

En simple : Le sélecteur de créneaux. Le prospect voit les jours disponibles sous forme de petits boutons horizontaux ("Mar 8 avr.", "Jeu 10 avr."). Il clique sur un jour et voit les heures disponibles ("14:00", "14:30", "15:00"). Il choisit une heure.

> Challenge : Sur mobile, il faut que les jours soient scrollables horizontalement sans que ça casse le design. Et les dates doivent être en français.

> Solution : Les dates sont des "chips" (petits boutons arrondis) dans un conteneur scrollable avec la scrollbar masquée. Les heures s'affichent en grille responsive (3-4 colonnes selon la largeur). Tout est formaté en français avec `date-fns` locale fr.

Fichiers clés : `components/SlotPicker.tsx` `components/DateChip.tsx` `components/TimeSlot.tsx`

6 Booking Submission FRONTEND + BACKEND

En simple : Le formulaire final. Le prospect entre son prénom et son email, confirme, et le système envoie la réservation. Ensuite, un bel écran de confirmation avec un checkmark vert et tous les détails du rendez-vous.

> Challenge : Si deux personnes bookent le même créneau exactement au même moment, il ne faut pas que les deux réussissent. Et si le créneau est pris, le message d'erreur doit être clair et proposer de choisir un autre créneau.

> Solution : Le serveur re-vérifie Google Calendar au moment de la soumission (pas juste à l'affichage). Si le créneau a été pris entre-temps, le prospect voit "Ce créneau vient d'être réservé" avec un bouton pour revenir au choix des créneaux. La page gère 4 états : checklist → choix → formulaire → confirmation.

Fichiers clés : `components/BookingForm.tsx` `components/BookingConfirmation.tsx`

7 Notifications & Email

API EXTERNES

En simple : Quand quelqu'un booke, deux choses se passent automatiquement : 1) Tu reçois un message Telegram avec le nom, l'email et l'heure du RDV. 2) Le prospect reçoit un email de confirmation avec tous les détails + un fichier .ics qu'il peut cliquer pour ajouter le RDV à son calendrier.

> Challenge : Si l'envoi de l'email ou du message Telegram échoue (serveur Brevo en panne, bot Telegram mal configuré), il ne faut PAS que ça casse la réservation du prospect. Le booking doit toujours fonctionner.

> Solution : Pattern "fire-and-forget" avec `Promise.allSettled`. Le système envoie les notifications en parallèle APRÈS avoir confirmé le booking. Si une notification échoue, l'erreur est loguée mais le prospect voit quand même sa confirmation. Le fichier .ics est généré côté serveur au format iCalendar standard.

Fichiers clés : `lib/notifications/telegram.ts` `lib/notifications/email.ts` `lib/notifications/ics.ts`

8 Dashboard & Reminders ADMIN + CRON

En simple : Ton espace admin. Tu te connectes sur /dashboard avec un mot de passe, et tu vois : tes RDV à venir, tes anciens RDV, et tu peux configurer tes plages horaires (ajouter/supprimer des disponibilités). En bonus : un job automatique qui tourne toutes les heures et envoie un email de rappel aux prospects 24h avant leur RDV.

> **Challenge :** Il fallait passer d'une config de disponibilité en dur dans le code à une config dynamique stockée en base de données, sans casser l'API existante.

> **Solution :** Nouvelle table `availability_windows` dans la base de données. L'API /api/slots lit d'abord la config en base ; si rien n'est configuré, elle utilise la config par défaut. Le dashboard est en thème sombre (pour toi, pas les prospects) avec une interface simple pour gérer les disponibilités. Le cron de rappel tourne via Vercel Cron (gratuit) toutes les heures.

Fichiers clés : `dashboard/page.tsx` `dashboard/availability/page.tsx` `api/cron/reminders/route.ts` `middleware.ts`

Stack technique

Next.js 15 Framework web	**Turso** Base de données	**Brevo** Emails
Telegram Notifications		

Ce qu'il te reste à faire

1. Google Cloud Setup (obligatoire)

1. Crée un projet sur Google Cloud Console
2. Active l'API Google Calendar
3. Crée des identifiants OAuth 2.0 (type: application de bureau)
4. Exécute le script : `npx tsx scripts/get-refresh-token.ts`
5. Copie le refresh token dans ton fichier `.env.local`

2. Turso Database (obligatoire)

1. Crée un compte sur turso.tech (gratuit)
2. Crée une base de données
3. Copie l'URL et le token dans `.env.local`
4. Exécute : `npx drizzle-kit push` pour créer les tables

3. Telegram Bot (obligatoire)

1. Parle à @BotFather sur Telegram
2. Crée un bot et récupère le token
3. Envoie un message à ton bot, récupère ton chat_id
4. Ajoute TELEGRAM_BOT_TOKEN et TELEGRAM_CHAT_ID dans `.env.local`

4. Variables d'environnement

Remplis toutes les variables dans `.env.local` :

- `TURSO_DATABASE_URL` / `TURSO_AUTH_TOKEN`
- `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` / `GOOGLE_REFRESH_TOKEN` / `GOOGLE_CALENDAR_ID`
- `TELEGRAM_BOT_TOKEN` / `TELEGRAM_CHAT_ID`

- `BREVO_API_KEY` (tu l'as déjà : xkeysib-844...)
- `DASHBOARD_PASSWORD` (choisis un mot de passe)
- `CRON_SECRET` (une chaîne aléatoire)

5. Déploiement Vercel

1. Pousse le repo sur GitHub
2. Connecte le repo à Vercel
3. Ajoute toutes les variables d'environnement dans Vercel
4. Pointe ton domaine remidene-enzo vers Vercel (DNS Squarespace)
5. Teste la page de booking en prod

Généré automatiquement par Claude — 6 avril 2026
Projet : /Users/Enzo/calendlybyEnzo/ · 8 phases · 21 requirements · 32 fichiers